Cloud Computing

Exercise - 9 Total points: 10

---

1. Create separate pods for Flask and PostgreSQL in Kubernetes

Instead of having a single pod with 2 containers for Flask and PostgreSQL, now create two separate pods, with the respective containers and configure them to achieve the desired communication and storage of our "Message-Board flask application".
So, we define two .yaml files, one for each service specifically.

- postgres-deployment.yaml
  There is a provision for Persistent Storage in Kubernetes which is configured in this file for PostgreSQL. Persistent Volume Claim (PVC) – it requests 1 GB of persistent storage which ensures the data is retained even if the pod restarts. It allows only one pod to mount the storage with ReadWriteOnce being configured. ClusterIP service is also configured. Note that the kind for this file is "Deployment" instead of our earlier "Pod".

Content of postgres-deployment.yaml file:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
        - ReadWriteOnce
  resources:
        requests:
         storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  replicas: 1
  selector:
        matchLabels:
        app: postgres
  template:
        metadata:
        labels:
         app: postgres
        spec:
        containers:
        - name: postgres
        image: postgres:14.1-alpine
        env:
        <Set the environment list for POSTGRES_USER, POSTGRES_PASSWORD and POSTGRES_DB>
        volumeMounts:
        - mountPath: /var/lib/postgresql/data
        name: postgres-storage
        ports:
```

```
            - containerPort: 5432
            volumes:
            - name: postgres-storage
            persistentVolumeClaim:
            claimName: postgres-pvc
---
apiVersion: v1
kind: Service
metadata:
  name: postgres
spec:
  selector:
            app: postgres
  ports:
            - protocol: TCP
            port: 5432
            targetPort: 5432
```

- flask-deployment.yaml –
  This sets up the flask app deployment (in a pod) and exposes the Flask service via NodePort. NodePort exposes the service externally on a random, valid high port number (30000 to 32767) on each node. We also specify the image which needs to be pulled from Docker Hub. It connects to PostgreSQL via DATABASE_URL and listens on port 5000. It also maps the port NodePort to Flask's port 5000. Flask running in the pod connects to PostgreSQL using the postgres service name.

Content of flask-deployment.yaml file:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app
spec:
  replicas: 1
  selector:
    matchLabels:
            app: flask-app
  template:
    metadata:
            labels:
             app: flask-app
    spec:
     containers:
            - name: flask-app
             image: <Put your docker image from docker hub>
             env:
              - name: DATABASE_URL
              value: "postgresql://postgres:srirama@postgres:5432/postgres"
              ports:
              - containerPort: 5000
---
apiVersion: v1
```

```
kind: Service
metadata:
  name: flask-service
spec:
  selector:
        app: flask-app
  ports:
        - protocol: TCP
        port: 80
        targetPort: 5000
  type: NodePort
```

TODO (Exercise)**:** (2 points)

a) Apply PostgreSQL deployment. Notice that you blindly cannot copy the YAML files. You need to correct the indentation logically.
b) Wait until the postgres pod is in running state.
c) Apply Flask app deployment
d) Verify deployments and show that the pods and services are running.
e) Check that the Flask app is running on localhost (127.0.0.1:5000) inside the container.
f) See that the message board is running as expected and as shown in the following screenshot.
g) NB! Show the commands and the screenshots
h) NB! At the end, delete the pods.
   - kubectl delete -f postgres-deployment.yaml
   - kubectl delete -f flask-deployment.yaml



Different IPs to access Flask Service:

1. http://localhost:5000 (Inside the Container Only, only the processes within the pod or the container can access it. To check, exec into the container with kubectl exec -it <pod_name> -- bin/sh and try curl http://localhost:5000
2. http://10.xx.xx.xx:5000 (Pod IP, Accessible within the Node, this IP is assigned by Kubernetes network to the Flask pod. Other pods within the network can access it but not externally.

3. http://<NODE_IP>:30448 (Accessible from the Whole Network) to obtain this port, check kubectl get services or kubectl get svc to see which NodePort is getting bound. Any machine inside the Kubernetes cluster or within the same network can reach it via http://<NODE_IP>:30448

2. Kubernetes Load Balancing and Horizontal Scaling

Kubernetes Task Details

In this task, multiple pod instances are created in Kubernetes where there is a Flask deployment with multiple Flask pods to distribute the load across the replicas. One pod is meant for a PostgreSQL relational database container to store the data, and one pod is for Nginx, which acts as a reverse proxy and load balancer.

Main Goal: Nginx receives the incoming HTTP traffic and distributes that traffic evenly across the Flask pods (via the Flask service). On initialization, 3 replicas of the Flask pods are created, ensuring that even if one pod crashes or fails, Kubernetes can automatically restart it using self-healing for high availability.

This exercise demonstrates **horizontal scaling** (where Kubernetes scales at the pod level). Kubernetes handles increased traffic by creating more replicas of the Flask pods, allowing load to be distributed across multiple pods.

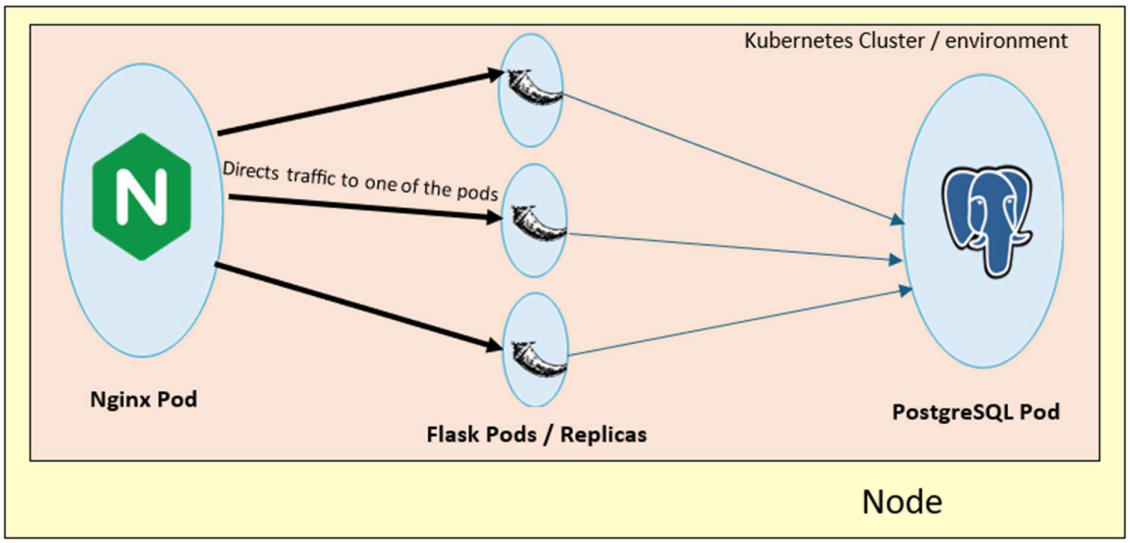
Why are multiple pods created for the Flask service instead of a single pod with multiple containers or multiple pods with multiple containers?

This is because Nginx can distribute traffic across multiple pods, not multiple containers in the same pod.

Nginx can be used as a load balancer in Kubernetes which distributes traffic across multiple pods (replicas) of a service. This is achieved through Kubernetes services, which abstract and load balance traffic to all pods matching specific labels. As Kubernetes ensures that each pod has its own unique IP address, it enables Nginx to balance traffic across these IPs. (Reference: https://www.f5.com/company/blog/nginx/load-balancing-kubernetes-services-nginx-plus)

In case of single pod with multiple containers, containers within the same pod can share volumes and network-related resources. Hence, traffic between these containers does not require external load balancing and they can directly communicate via localhost. Also, if the pod crashes, all the containers also get affected.

The final setup is as follows:



We require the following configuration and files to setup the required pods and microservices. They are provided in the Moodle.

NB! You must adapt those files with your own flask image from Dockerhub and passwords

1. flask.yaml file for deploying the Flask pods as Deployment and a Service

   What it does: Three flask pods / replicas are created via a Deployment and they are accessible through a Kubernetes Service. A Deployment allows to define the desired state of the application, manage identical pods, and handle the orchestration of changes to achieve the required state. If a pod crashes, Kubernetes will automatically create a new one (self-healing). The Flask pods are exposed internally as a ClusterIP service named flask-service. Service is used to allow communication between pods inside the cluster.

2. postgres.yaml file for deploying PostgreSQL as a StatefulSet (since database needs to have persistent storage facility)

   What it does: A PersistentVolumeClaim is configured for creating a persistent storage resource which ensures that PostgreSQL data is not lost if the pod is deleted or rescheduled. A StatefulSet feature is used to manage stateful applications since data persistence and consistent network identity are important for a database (even if the pod restarts, it still gets attached to the same PVC). Service makes PostgreSQL reachable within the cluster using the DNS name postgres-service. clusterIP is set to None to allow direct pod-to-pod communication and preserve the stateful network identity needed for database consistency and replication.

3. nginx.conf file for configuring Nginx as a reverse proxy and as a load balancer which will enable distribution of traffic to the pods.

   What it does: This is a configuration file for NGINX. The events section is a necessary part of the configuration file for handling event-based processing for connections and should be mentioned even if it is empty (i.e., no connection settings are required for this exercise). The http block configures how to handle web traffic. The upstream section creates a cluster of backend servers named flask_backend, which directs NGINX to forward requests to the Flask service on the internal port 5000 in the Kubernetes cluster. The server block listens for HTTP traffic on the default port 80. The location block defines how NGINX should handle incoming HTTP requests that match the root path of the URL or start with /. NGINX modifies or adds headers when forwarding the request

to Flask, which allows Flask to know the original host used by the client, the real client IP, and the chain of proxies involved in the request.

Note: This file is not provided to you. It is actually inserted in the nginx.yaml

4. nginx.yaml file for deploying Nginx as a pod and exposing it as a service.

What it does: This file defines a Nginx ConfigMap, Deployment, and Service in Kubernetes. The ConfigMap is used to store configuration data, where it contains nginx.conf file as the key-value data. The Deployment defines how Nginx should run within the Kubernetes cluster. A volumeMounts section is used to mount nginx.conf file. The Service exposes Nginx to external traffic using a LoadBalancer.

Resources:

a. https://www.digitalocean.com/community/tutorials/understanding-the-nginx-configuration-file-structure-and-configuration-contexts
b. https://medium.com/@ruchirthaman/practical-implementation-of-a-load-balancer-using-nginx-and-flask-9562e2ea5095
c. https://www.solo.io/topics/nginx/nginx-configuration)

Deploying the complete setup

- First, apply the PostgreSQL yaml file to create the pod
  kubectl apply -f postgres.yaml
- Once the pod is in Running state(use kubectl get pods -o wide to check the status) , apply the flask yaml file to create the pods for Flask application
  kubectl apply -f flask.yaml
- Once the pods are in Running state, apply the nginx yaml file to create the Nginx pod
  kubectl apply -f nginx.yaml

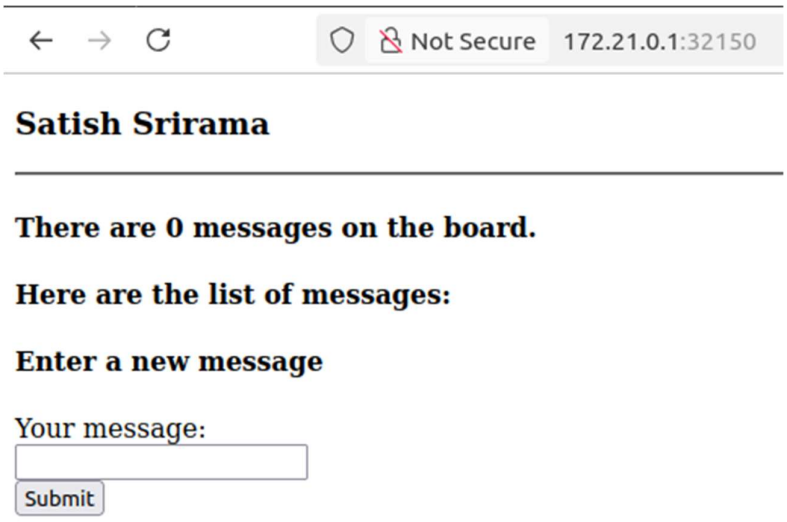
You should be seeing the following pods

```
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale$ kubectl get pods
NAME                                READY   STATUS    RESTARTS      AGE
flask-deployment-6dc4d49fdf-7dl5r   1/1     Running   0             53s
flask-deployment-6dc4d49fdf-lxnc2   1/1     Running   0             53s
flask-deployment-6dc4d49fdf-wxs2b   1/1     Running   1 (46s ago)   53s
nginx-deployment-c57f54b7f-2xpcj    1/1     Running   0             17s
postgres-0                          1/1     Running   0             76s
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale$
```

- Check the services in Kubernetes and whether the PersistentVolumeClaim for PostgreSQL is bound correctly:
  - kubectl get svc or kubectl get services
  - kubectl get pvc

```
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale$ kubectl get services
NAME               TYPE           CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
flask-service      ClusterIP      10.43.59.1    <none>        5000/TCP         2m17s
kubernetes         ClusterIP      10.43.0.1     <none>        443/TCP          3h28m
nginx-service      LoadBalancer   10.43.15.93   <pending>     80:32150/TCP     100s
postgres-service   ClusterIP      None          <none>        5432/TCP         2m39s
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale$
```

The UI of home.html is accessible externally as http://<node_ip>:32150 as nginx-service's internal port 80 is mapped to external port 32150 (exposed via NodePort configured for flask.yaml)



- Now enter a message



- The message is handled by the nginx loadbalancer, which has routed the message to one of the flask pods
- Check the logs of all the pods (to ensure the functionality is as expected):

kubectl logs -f <pod_name>

```
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale$ kubectl logs -f nginx-deployment-c5
7f54b7f-2xpcj
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform
configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.
sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/de
fault.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/
default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
10.42.0.1 - - [05/Apr/2025:08:02:32 +0000] "GET / HTTP/1.1" 200 537 "-" "Mozilla/5.
0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0"
10.42.0.1 - - [05/Apr/2025:08:02:33 +0000] "GET /favicon.ico HTTP/1.1" 404 207 "htt
p://172.21.0.1:32150/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/201
00101 Firefox/136.0"
10.42.0.1 - - [05/Apr/2025:08:03:49 +0000] "GET /?msg=With+proper+usage+of+Nginx HT
TP/1.1" 200 678 "http://172.21.0.1:32150/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64;
 rv:136.0) Gecko/20100101 Firefox/136.0"
```

```
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale$ kubectl logs -f flask-deployment-6d
c4d49fdf-7dl5r
 * Serving Flask app 'app.py' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Us
e a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.42.0.68:5000
Press CTRL+C to quit
10.42.0.71 - - [05/Apr/2025 08:02:33] "GET /favicon.ico HTTP/1.0" 404 -
10.42.0.71 - - [05/Apr/2025 08:03:49] "GET /?msg=With+proper+usage+of+Nginx HTTP/1.
0" 200 -
```

- You can observe the message is handled by the above pod.

```
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale$ kubectl logs -f flask-deployment-6d
c4d49fdf-lxnc2
 * Serving Flask app 'app.py' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Us
e a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.42.0.70:5000
Press CTRL+C to quit
```

```
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale$ kubectl logs -f flask-deployment-6d
c4d49fdf-wxs2b
 * Serving Flask app 'app.py' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Us
e a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.42.0.69:5000
Press CTRL+C to quit
10.42.0.71 - - [05/Apr/2025 08:02:32] "GET / HTTP/1.0" 200 -
```

Notice that the flask application is also accessible at http://10.42.0.68:5000, http://10.42.0.69:5000 and http://10.42.0.70:5000 from the same node. However, with the horizontal scaling you will not expose these interfaces and all clients will approach only through http://<node_ip>:32150 from any node in this network.

You can enter 5 more messages.

- Show the screenshot of your messages page (1 point)
- Show which of the pods are handling those messages and the logs of nginx. (1 point)
- Confirm that the messages are correctly stored in the PostgreSQL database wrt to the pod. Connect to the DB and show that the messages are actually getting stored on the single DB, even though they are handled by multiple flask application pods. (1 point)

  Show the commands and the screenshots.

- To send heavy-volume of messages via Nginx as the load-balancer

for i in {1..100}; do curl -s "http://172.21.0.1:32150/?msg=Hello_$i"; done

- Check and show how the command distributes the messages to various flask pods. (1 point)
- In this setup, the default ClusterIP service (flask-service) performs round-robin load-balancing at the TCP level and when Nginx forwards a request to flask-service:5000, Kubernetes internally balances it across the three Flask pods.

3. Generating the load with Apache JMeter

To generate many user requests, **Apache JMeter**, a Java-based framework for simulating users and generating web traffic can be used.

Note: Preferably delete the existing pods and re-deploy them for fresh logs monitoring.

Steps to be done in Apache JMeter:

1. Ensure Java version > 8 (In this case, Java 21 was used)
2. Download JMeter using wget https://dlcdn.apache.org//jmeter/binaries/ apache-jmeter-5.6.3.zip
3. unzip apache-jmeter-5.6.3.zip
4. cd apache-jmeter-5.6.3/bin

5. java -jar ApacheJMeter.jar (This will open the UI for Apache JMeter)
6. Now, perform the following steps:

- Right Click on Test Plan -> Add -> Threads (Users) -> Thread Group
  Here, we configure the number of threads as 10
  Change the Loop count to 20. (will account for 10 users, sending 20 messages each)
  Uncheck the option of Same user on each iteration (so that we can observe different user instances).
- Right Click on Test Plan -> Add -> Config Element -> User Defined Variable
  Here, Click on Add -> Name as counter, Value as 1
- Right Click on Thread Group -> Add -> Config Element -> Counter
  Here, choose Starting value as 1
  Increment as 1
  Maximum value as 20
  Exported Variable Name as counter
  Also, check on track counter independently for each user.
- Right Click on Thread Group -> Sampler -> HTTP Request
  For server name or ip, use ip of node: 192.168.0.119
  For port number, use port which allows external access (check via kubectl get svc): 32197
  Keep the method as GET, and for PATH, use /
  For Parameters, Click on Add -> Name as msg, Value as
  Hello_${counter}_from_user_${__threadNum}
  ${__threadNum} is a built-in JMeter function that dynamically assigns a number to each thread (user) based on the "Number of Threads (Users)" in the Thread Group.
  Right Click on Thread Group -> Add -> Listener -> View Results Tree
- Right Click on Thread Group -> Add -> Listener -> Response Time Graph
  Here, change the Interval (ms) value to 1000

Once this is done, go to File -> Save Test Plan as -> load_test.jmx

Then, click on the Green Run button and view the result visualization view.

TODO (Exercise)**:**

a) Show the screenshots of your Apache JMeter configurations. NB! The screenshots should show you yourself have performed the exercise. (1 point)
b) Show the screenshot of your messages page
c) Show which of the pods are handling those messages and the logs of nginx. (1 point)

4. Working with HorizontalPodAutoscaler in Kubernetes

To consider the Scalability aspect for Kubernetes, HorizontalPodAutoscaler (HPA) scales the workload according to the demand. With an increase in the load, a greater number of pods are deployed to meet the requirement and if the load decreases, and the number of pods is above the configured minimum, then it scales down the workload. (Reference: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/)

First, delete all the existing pod deployments. A new flask yaml file is created to specify the cpu and memory requirements for the flask pods.

flask_hpa.yaml is provided in the Moodle. NB! You must adapt it with your own flask image from Dockerhub and passwords.

Now load all the pods, deployments etc. i.e. postgres.yaml, nginx.yaml and flask_hpa.yaml in the logical order, which you followed earlier.

HPA is not enabled in Kubernetes by default.

- To create HPA for the flask pods based on the CPU usage:

kubectl autoscale deployment flask-deployment --min=3 --max=10 --cpu-percent=60

This will ensure a minimum of 3 flask pods and a maximum of 10 flask pods where, the scaling gets activated when the CPU usage is above 60%. The HPA controller will increase and decrease the number of replicas (by updating the Deployment) to maintain an average CPU utilization across all Pods of 60%.

Note: The CPU usage is configurable. A good number is 70%. However, I could not generate enough load to force autoscale on my laptop. Therefore, reduced the value to 60%.

- To check the status of the newly-made HorizontalPodAutoscaler:
  kubectl get hpa
- To check the deployment:
  kubectl get deployment flask-deployment

```
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale$ kubectl get hpa
NAME               REFERENCE                     TARGETS              MINPODS
MAXPODS   REPLICAS   AGE
flask-deployment   Deployment/flask-deployment   cpu: <unknown>/70%   3
10        0          6s
```

NB! In case the CPU usage is shown as <unknown> here, it indicates that there is an issue with the metrics server which collects CPU and memory usage from kubelet.

In this case, the existing metrics-server can be deleted: kubectl delete -f components.yaml -n kube-system and reinstalled from the GitHub repository with

kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

Note: Usually components.yaml is not downloaded so the delete is not necessary

Then, check if the metrics server is working properly:
kubectl get deployment metrics-server -n kube-system

```
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale/hpa$ kubectl get hpa
NAME               REFERENCE                     TARGETS       MINPODS   MAXPODS
   REPLICAS   AGE
flask-deployment   Deployment/flask-deployment   cpu: 1%/70%   3         10
   3          6m1s
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale/hpa$ kubectl get deployment flask
-deployment
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
flask-deployment   3/3     3            3           14m
```

- In a separate terminal, start a load generator to simulate high traffic (this also creates a load-generator pod in Kubernetes)
  kubectl run -i --tty load-generator --rm --image=busybox:1.28 --restart=Never -- /bin/sh -c "while sleep 0.001; do wget -q -O- http://flask-service:5000; done"

It repeatedly sends HTTP GET requests to the Flask service every 0.001 seconds (sends 1000 requests per second) to increase CPU load. As it opens in the interactive terminal mode, it keeps displaying the content of home.html (which is displayed via the flask-service)

- Now, to verify the load, use the command kubectl get hpa flask-deployment –watch

As the CPU load exceeds 60%, additional pods / replicas are created which shows the horizontal scalability nature that can be achieved with Kubernetes.

```
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale/hpa$ kubectl get hpa flask-deploy
ment --watch
NAME               REFERENCE                     TARGETS              MINPODS
MAXPODS   REPLICAS   AGE
flask-deployment   Deployment/flask-deployment   cpu: <unknown>/60%   3
10        4          11m
flask-deployment   Deployment/flask-deployment   cpu: 47%/60%         3
10        4          11m
```

The creation of additional pods can be verified using kubectl get pods -o wide:

```
osboxes@osboxes:~/satish/CC/kube/ex9kube/scale/hpa$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS      AGE   IP
      NODE      NOMINATED NODE   READINESS GATES
flask-deployment-549cdc6f99-c7kz9   1/1     Running   0             35m   10.42.
0.76   osboxes   <none>           <none>
flask-deployment-549cdc6f99-d6xbp   1/1     Running   1 (35m ago)   35m   10.42.
0.75   osboxes   <none>           <none>
flask-deployment-549cdc6f99-fckwl   1/1     Running   1 (35m ago)   35m   10.42.
0.77   osboxes   <none>           <none>
flask-deployment-549cdc6f99-hmw29   1/1     Running   0             6s    10.42.
0.83   osboxes   <none>           <none>
load-generator                      1/1     Running   0             69s   10.42.
0.82   osboxes   <none>           <none>
nginx-deployment-c57f54b7f-rxnkx    1/1     Running   0             35m   10.42.
0.78   osboxes   <none>           <none>
postgres-0                          1/1     Running   0             36m   10.42.
0.74   osboxes   <none>           <none>
```

Observe a 4th flask-deployment-xxx is created.

You stop the load and eventually that extra pod will be removed.

```
^Cosboxes@osboxes:~/satish/CC/kube/ex9kube/scale/hpa$ kubectl get hpa flask-depl
oyment --watch
NAME               REFERENCE                     TARGETS       MINPODS   MAXPODS
   REPLICAS   AGE
flask-deployment   Deployment/flask-deployment   cpu: 1%/60%   3         10
   4          16m
flask-deployment   Deployment/flask-deployment   cpu: 1%/60%   3         10
   4          17m
flask-deployment   Deployment/flask-deployment   cpu: 1%/60%   3         10
   3          17m
```

Instead of manually terminating the load-generator and to prevent multiple pods being created excessively, we can use the following configuration. If the CPU load is above 60 %, scale up by creating a Flask pod, then scale down as the load decreases. Here we can even change minRelicas and set it to 1, to see autoscaling behaviour more clearly.

First, use kubectl delete -f flask_hpa.yaml to delete the pods and execution.

For this, you have to create a new file flask_hpa_v1.yaml (to the same content of flask_hpa.yaml, add this at the end):

```
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: flask-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: flask-deployment
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
      policies:
      - type: Pods
        value: 1
        periodSeconds: 15
    scaleDown:
      stabilizationWindowSeconds: 30
      policies:
      - type: Pods
        value: 1
        periodSeconds: 15
```

In the HPA part, If CPU > 60%, it adds 1 pod every 15s and remove 1 pod every 15s if the CPU load goes very low. This also prevents rapid scaling changes by waiting 30s before making new scaling decisions. This has a specific configuration for HPA unlike the earlier version where Kubernetes decided the behaviour.

- kubectl apply -f flask_hpa_v1.yaml
- kubectl get hpa

In a separate terminal, start a load generator to simulate high traffic.

TODO (Exercise)**:** (2 points)

1. Now verify the load and demonstrate the autoscaling is working as it is expected.
2. NB! Show the commands and the screenshots
3. NB! At the end, delete the pods.

Follow these guidelines while submitting the exercises

1. Put a pdf file with your answers and screenshots for the questions and name it as your studentid_QE9
2. The screenshots are meant to show that the work is done and by you. So, have enough proofs in your answer sheet.
3. **Note**: Do not blindly copy the commands. Try to understand them. The provided commands are to guide you with the initial process.
4. Do not copy code from internet. If you copy, please leave a comment with the URL on that code/answer sheet
5. Plagiarism will not be tolerated and dealt through the formal channels of the university